

CO5 UNIT 5 – ACTIVITY 1

Machine Learning Model Comparison Using Matplotlib

Aim

To implement and compare different Machine Learning algorithms using Python and Matplotlib, and visualize their performance using accuracy comparison charts and different Matplotlib plotting functions.

Objectives

- To implement multiple Machine Learning algorithms.
- To calculate and compare their accuracy.
- To visualize model performance using Matplotlib.
- To demonstrate Matplotlib functions such as colors, grids, labels, legends, markers, annotations, and different chart types.
- To identify the most efficient model based on accuracy.

Algorithms Used

1. Logistic Regression
2. Decision Tree Classifier
3. Random Forest Classifier
4. K-Nearest Neighbors (KNN)
5. Support Vector Machine (SVM)

Dataset

Iris Dataset

The Iris dataset contains measurements of iris flowers:

- Sepal Length
- Sepal Width

- Petal Length
- Petal Width

The target contains three flower classes:

- Setosa
- Versicolor
- Virginica

CODE

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# -----
# 1. Load Dataset
# -----

iris = load_iris()

X = iris.data
y = iris.target

# -----
# 2. Split Dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```

# -----
# 3. Feature Scaling
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# 4. Create Machine Learning Models
# -----

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "SVM": SVC(kernel="linear")
}

# -----
# 5. Train Models and Calculate Accuracy
# -----

model_names = []
accuracies = []
predictions = {}

for name, model in models.items():

    # Use scaled data for Logistic Regression, KNN and SVM
    if name in ["Logistic Regression", "KNN", "SVM"]:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    model_names.append(name)
    accuracies.append(accuracy * 100)
    predictions[name] = y_pred

print("-----")
print("Model:", name)
print("Accuracy:", round(accuracy * 100, 2), "%")

```

```

# -----
# 6. Print Final Comparison
# -----

print("\n===== MODEL COMPARISON =====")

for name, accuracy in zip(model_names, accuracies):
    print(f"{name:<25} : {accuracy:.2f}%")

best_index = np.argmax(accuracies)

print("\nBest Model:", model_names[best_index])
print("Best Accuracy:", round(accuracies[best_index], 2), "%")

# =====
# MATPLOTLIB VISUALIZATION
# =====

# -----
# 7. Bar Chart
# -----

plt.figure(figsize=(10, 6))

bars = plt.bar(
    model_names,
    accuracies,
    color=[
        "skyblue",
        "orange",
        "green",
        "purple",
        "red"
    ],
    edgecolor="black"
)

plt.title(
    "Machine Learning Model Accuracy Comparison",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Machine Learning Models", fontsize=12)
plt.ylabel("Accuracy (%)", fontsize=12)

plt.ylim(80, 105)

plt.grid(
    axis="y",

```

```

        linestyle="--",
        alpha=0.7
    )

plt.xticks(rotation=20)

# Display accuracy on top of bars
for bar, accuracy in zip(bars, accuracies):

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.5,
        f"{accuracy:.2f}%",
        ha="center",
        fontsize=10,
        fontweight="bold"
    )

plt.tight_layout()

plt.savefig("accuracy_bar_chart.png", dpi=300)

plt.show()

# -----
# 8. Line Graph
# -----

plt.figure(figsize=(10, 6))

plt.plot(
    model_names,
    accuracies,
    marker="o",
    linestyle="--",
    linewidth=2,
    markersize=8,
    label="Model Accuracy"
)

plt.title(
    "Accuracy Trend of Machine Learning Models",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Machine Learning Models")
plt.ylabel("Accuracy (%)")

plt.ylim(80, 105)

```

```

plt.grid(
    True,
    linestyle=":",
    alpha=0.7
)

plt.legend()

plt.xticks(rotation=20)

# Annotate highest accuracy
best_accuracy = accuracies[best_index]

plt.annotate(
    "Best Model",
    xy=(best_index, best_accuracy),
    xytext=(best_index, best_accuracy - 5),
    arrowprops=dict(
        arrowstyle="->",
        color="black"
    ),
    fontsize=11
)

plt.tight_layout()

plt.savefig("accuracy_line_chart.png", dpi=300)

plt.show()

# -----
# 9. Scatter Plot - Actual vs Predicted
# -----

best_model_name = model_names[best_index]

best_predictions = predictions[best_model_name]

plt.figure(figsize=(8, 6))

plt.scatter(
    range(len(y_test)),
    y_test,
    marker="o",
    label="Actual Values",
    s=70
)

plt.scatter(
    range(len(best_predictions)),
    best_predictions,

```

```

        marker="x",
        label="Predicted Values",
        s=70
    )

plt.title(
    f"Actual vs Predicted Values - {best_model_name}",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("Test Sample Number")
plt.ylabel("Class")

plt.grid(
    True,
    linestyle="--",
    alpha=0.6
)

plt.legend()

plt.tight_layout()

plt.savefig("actual_vs_predicted.png", dpi=300)

plt.show()

# -----
# 10. Confusion Matrix
# -----

cm = confusion_matrix(y_test, best_predictions)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
)

disp.plot(cmap="Blues")

plt.title(
    f"Confusion Matrix - {best_model_name}",
    fontsize=15,
    fontweight="bold"
)

plt.grid(False)
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=300)
plt.show()

```

Matplotlib Functions Implemented

Matplotlib Function	Purpose
<code>plt.figure()</code>	Creates a new figure
<code>plt.bar()</code>	Creates a bar chart
<code>plt.plot()</code>	Creates a line graph
<code>plt.scatter()</code>	Creates a scatter plot
<code>plt.title()</code>	Adds chart title
<code>plt.xlabel()</code>	Labels X-axis
<code>plt.ylabel()</code>	Labels Y-axis
<code>plt.grid()</code>	Displays grid
<code>plt.legend()</code>	Displays legend
<code>plt.xticks()</code>	Rotates/formats X-axis labels
<code>plt.ylim()</code>	Sets Y-axis range
<code>plt.text()</code>	Adds text to graph
<code>plt.annotate()</code>	Adds annotation and arrow
<code>plt.tight_layout()</code>	Adjusts graph spacing
<code>plt.savefig()</code>	Saves graph as image
<code>plt.show()</code>	Displays graph

<code>plt.color</code> <code>color=</code>	Changes plot colors
<code>marker=</code>	Changes point/line markers
<code>linestyle=</code>	Changes line style
<code>linewidth=</code>	Changes line thickness
<code>markersize=</code>	Changes marker size
<code>alpha=</code>	Controls transparency
<code>fontsize=</code>	Changes text size
<code>fontweight=</code>	Makes text bold

Procedure

Step 1-Import the required Python libraries such as NumPy, Matplotlib and Scikit-learn.

Step 2-Load the built-in Iris dataset using `load_iris()`.

Step 3-Separate the dataset into input features X and target variable y.

Step 4-Split the dataset into training and testing datasets using `train_test_split()`.

Step 5-Standardize the features using `StandardScaler()`.

Step 6-Create five Machine Learning models:

- Logistic Regression
- Decision Tree

- Random Forest
- KNN
- SVM

Step 7-Train each model using the training data.

Step 8-Predict the classes using the testing data.

Step 9-Calculate model accuracy using `accuracy_score()`.

Step 10-Store the accuracy of all models and compare their performance.

Step 11-Use Matplotlib to create:

- Bar chart
- Line graph
- Scatter plot
- Confusion matrix

Step 12-Add colors, grids, labels, legends, markers, annotations and titles to improve visualization.

Step 13-Save the generated charts as PNG files.

Output

Model Accuracy

```
-----  
Model: Logistic Regression  
Accuracy: 93.33 %  
-----
```

```
Model: Decision Tree  
Accuracy: 93.33 %  
-----
```

```
Model: Random Forest  
Accuracy: 90.0 %  
-----
```

```
Model: KNN  
Accuracy: 93.33 %  
-----
```

```
Model: SVM  
Accuracy: 100.0 %
```

```
===== MODEL COMPARISON =====  
Logistic Regression      : 93.33%  
Decision Tree            : 93.33%  
Random Forest           : 90.00%  
KNN                      : 93.33%  
SVM                     : 100.00%
```

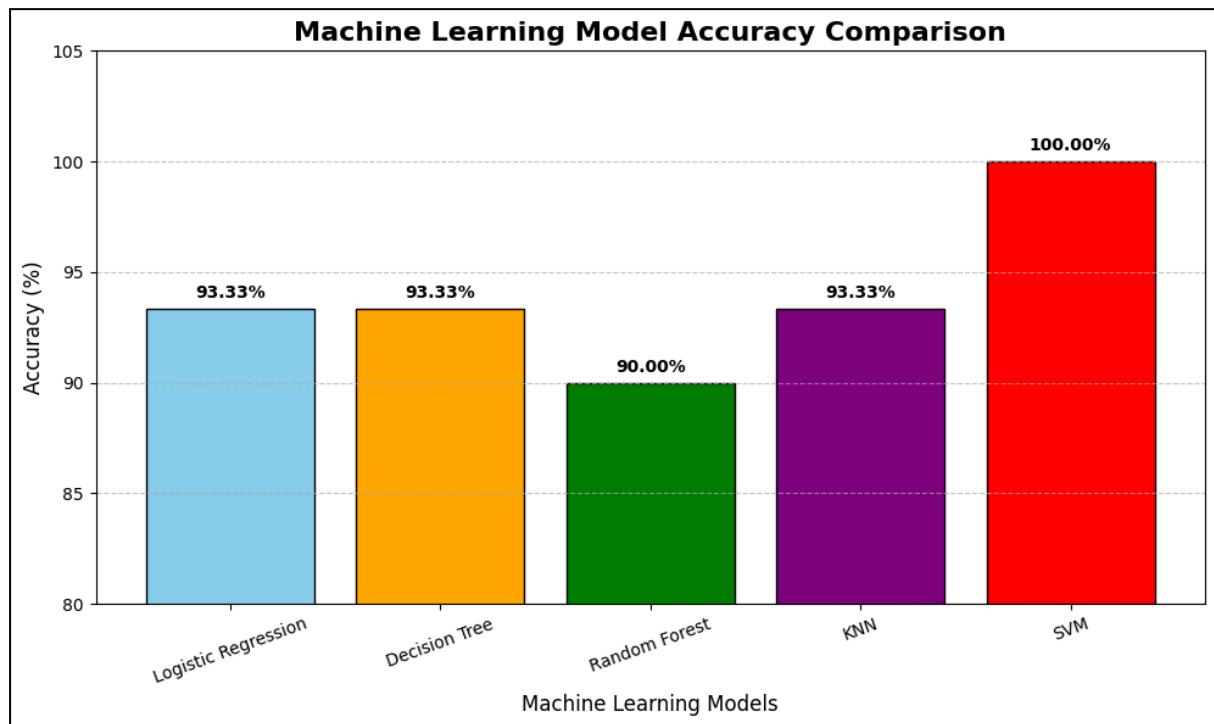
```
Best Model: SVM  
Best Accuracy: 100.0 %
```

Graph 1 – Accuracy Bar Chart

The bar chart compares the accuracy of all five Machine Learning algorithms.

Interpretation:

A higher bar represents better classification accuracy. The SVM model with the highest accuracy is considered the most effective model for this dataset.

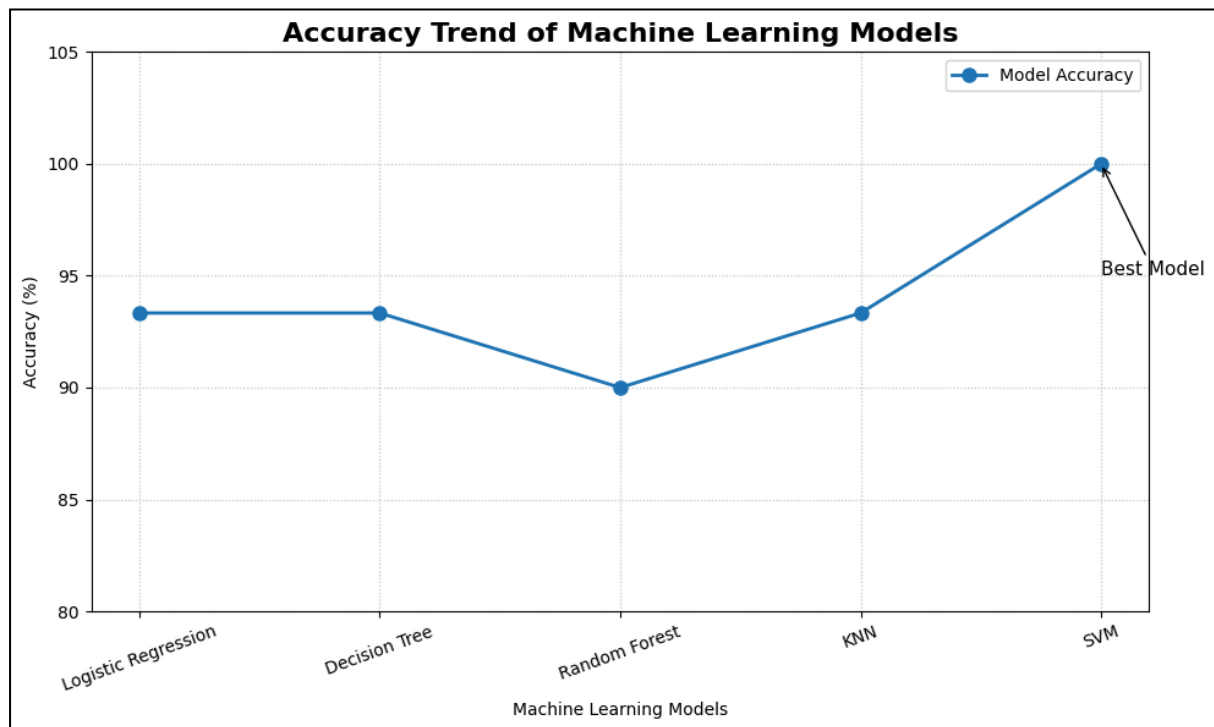


Interpretation:

A higher bar represents better classification accuracy. The SVM model with the highest accuracy is considered the most effective model for this dataset.

Graph 2 – Accuracy Line Graph

The line graph shows the accuracy trend across different Machine Learning models.

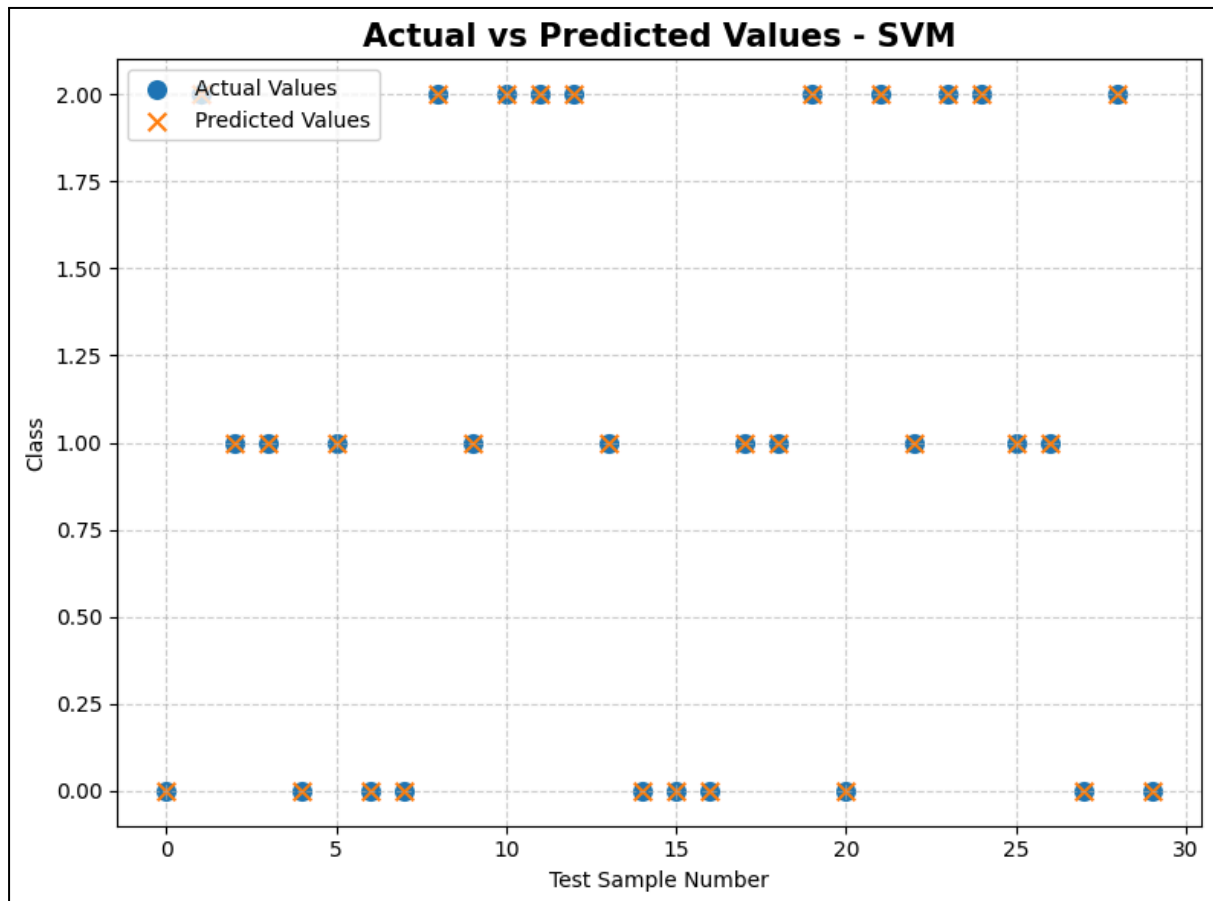


Interpretation:

The graph makes it easy to identify variations in model performance. Markers are used to clearly identify each model's accuracy.

Graph 3 – Actual vs Predicted Scatter Plot

The scatter plot compares the actual class values with the values predicted by the best-performing model.

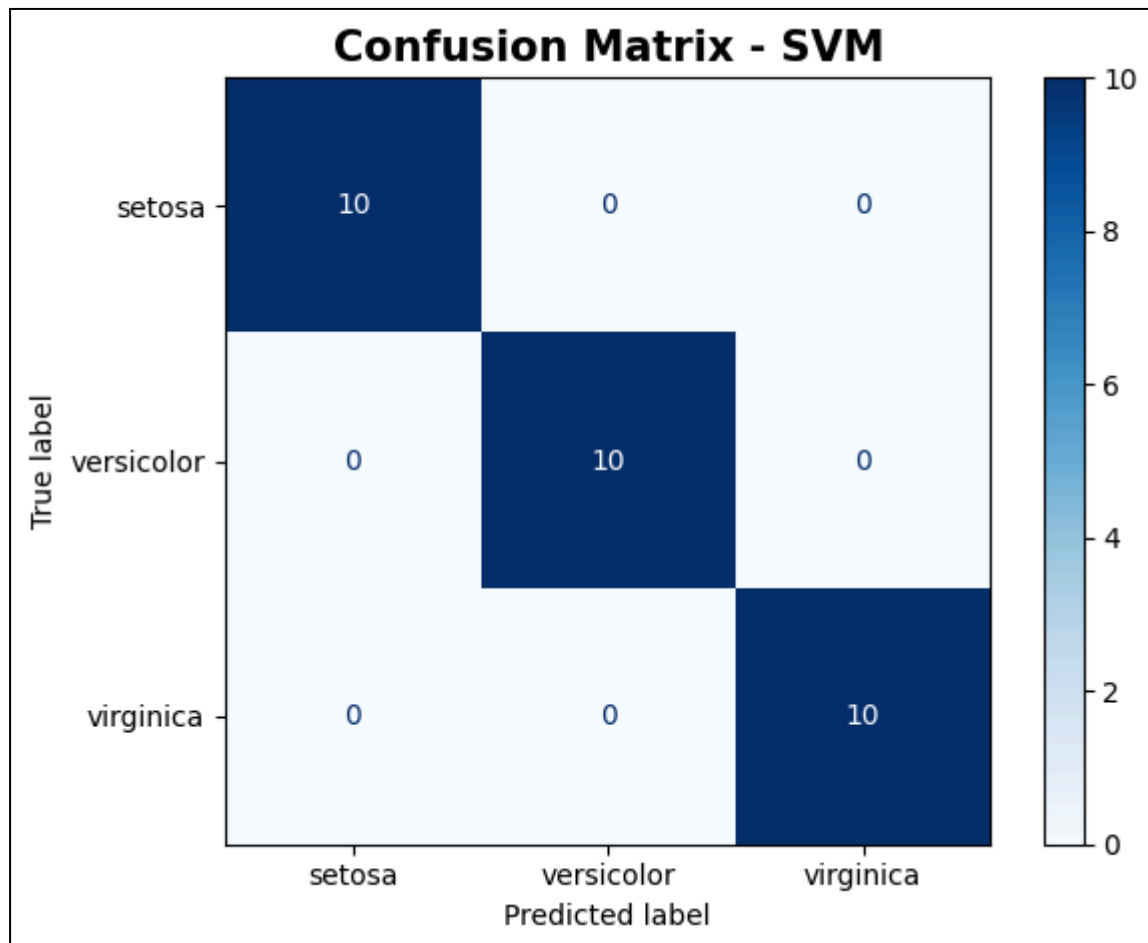


Interpretation:

When actual and predicted values are similar, the model is making correct predictions.

Graph 4 – Confusion Matrix

The confusion matrix displays the number of correct and incorrect predictions for each Iris class.



Interpretation:

Values along the diagonal represent correctly classified samples, while values outside the diagonal represent misclassifications.

Result

The performance of five Machine Learning algorithms was successfully compared using their classification accuracy. Matplotlib was used to create bar charts, line graphs, scatter plots, and a confusion matrix. Various visualization features such as colors, grids, markers, labels, legends, annotations, and titles were implemented.

The model SVM with the highest accuracy was identified as the best-performing model for the Iris dataset.

Conclusion

Thus, different Machine Learning algorithms were successfully implemented and compared using Python. Matplotlib provided effective visualization of model performance and made it easy to identify the most efficient algorithm.